

Document de conception — PertFlow

Ce document décrit l'**architecture** de PertFlow, ses **choix techniques** et leurs **justifications**. Il s'adresse à un développeur qui doit comprendre ou faire évoluer l'outil. Le compagnon maintenance.md détaille la reprise pratique et les pièges.

1. Objectif et contraintes structurantes

PertFlow est un outil de planification **PERT** en **application web locale** (un fichier HTML). Deux contraintes dictent toute l'architecture :

1. **Ouverture en `file://` par double-clic — PRIMORDIAL.** L'outil tourne sur des postes d'entreprise verrouillés par la DSI : **aucun serveur, aucun build, aucune architecture client-serveur**. Deux conséquences absolues : - **Pas de modules ES6** (`<script type="module">` + `import / export`) : ils forceraient un serveur (CORS en `file://`). Tout le code est chargé par des `<script src>` **classiques**, et vit dans le **scope global** (les fonctions `function pertX()` sont globales). - **Pas de `fetch()` /XHR de fichiers locaux** : les fichiers utilisateur sont lus via `<input type="file">` + `FileReader` , jamais par `fetch` .
 2. **Licence MIT uniquement** : toutes les bibliothèques sont locales (`lib/`) et sous licence permissive compatible MIT. Aucune dépendance réseau au runtime.
-

2. Vue d'ensemble et pile technique

Besoin	Choix	Licence	Fichier
Canvas / graphe	LiteGraph.js	MIT	<code>lib/litegraph.js</code> + <code>.css</code>
Export PDF	jsPDF	MIT	<code>lib/jspdf.umd.min.js</code>
Zip/dézip (import & export Excel)	fflate	MIT	<code>lib/fflate.min.js</code>
Export PNG	API Canvas2D native	—	—
UI	Vanilla JS + CSS custom	—	—

Aucun framework (React/Vue...), aucun bundler au sens classique : la simplicité prime, et la contrainte `file://` l'impose.

Structure des fichiers

```
pertflow/
├─ index.html          # Point d'entrée : DOM + <script src> de tout le code
├─ lib/                # Bibliothèques locales (LiteGraph, jsPDF, fflate)
├─ css/style.css       # Styles globaux (thème sombre)
├─ src/
│   ├─ nodes.js        # Types de nœuds PERT + rendu custom (LiteGraph)
│   ├─ pert_engine.js  # Calcul PERT, conversions dates↔unités, layout, chemin cri
│   │                  #   estimation charge/coût
│   ├─ ui.js           # Toolbar, panneau, dialogues, menus, filtre, barre de stat
│   ├─ storage.js      # Sérialisation/chargement .pert
│   ├─ history.js      # Undo/Redo par snapshots
│   ├─ autosave.js     # Filet anti-crash (snapshot localStorage)
│   ├─ align.js        # Boîte d'alignement / distribution des nœuds sélectionnés
│   ├─ t0_marker.js    # Repère T0 + bande « travaux anticipés » (2 couches de rer
│   ├─ time_grid.js    # Trame calendaire de fond, optionnelle et d'intensité régl
│   ├─ synthesis.js    # Fenêtre de synthèse (planification) + helpers de rendu pa
│   ├─ suivi.js        # Fenêtre de suivi d'avancement – réutilise ces helpers
│   ├─ import.js       # Fenêtre d'import + registre des formats (même patron que
│   ├─ import_excel.js # Import des .xlsml legacy CPERT (fflate + DrawingML)
│   ├─ import_pert.js  # Import/concaténation d'un .pert dans le planning courant
│   ├─ export.js       # Fenêtre d'export + PNG/PDF + helpers de téléchargement
│   ├─ export_csv.js   # Export CSV
│   ├─ export_xlsx.js  # Mini-writer XLSX générique (sur fflate)
│   ├─ export_gantt.js # Gantt chargé (Excel) + MS Project (MSPDI XML)
│   └─ export_microjalons.js # Micro-jalonnement (Excel)
├─ link_routing.js     # Rendu des liens : styles + routage orthogonal (évitement)
├─ docs/               # Manuel, conception, maintenance, notes de version (MD + f
├─ tools/              # Suite de tests + captures d'écran (cf. tools/README.md)
├─ test_cases/         # Jeux d'essai – versionnés en LISTE BLANCHE (cf. tools/RE
├─ scripts/
│   ├─ build-bundle.js # Génère le fichier autonome dist/pertflow.html
│   └─ make-release.js # Fabrique l'archive de livraison (bundle + manuel + notes)
└─ dist/pertflow.html  # Livrable autonome (versionné)
```

L'ordre de chargement des `<script>` dans `index.html` matérialise les dépendances (chaque module s'appuie sur les globales définies avant lui ; `ui.js` est chargé en dernier et fait le câblage).

3. Modèle de données

Le fichier `.pert`

```
{
  "version": "1.0",
  "meta": { "title", "t0", "unit", "layout_gap", "link_mode", "prop_width",
            "hours_per_month", "hours_per_day", "hourly_rate", "groups", "autosave" }
  "graph": { /* sérialisation LiteGraph native (graph.serialize()) */ }
}
```

- `meta.unit` $\in$ `"j" | "sem" | "mois"`.
- `meta.groups` = registre `{ nom_de_groupe: couleur }` (mémoire des couleurs de groupe).
- Les **valeurs calculées** (ES/EF/LS/LF/slack/is_critical, coût) **ne sont PAS sérialisées** : elles sont **recalculées** au chargement (`pertRecalc`) → cohérence garantie même sur un vieux fichier. Les propriétés **saisies** vivent dans `node.properties` (sérialisées nativement par LiteGraph).

Les nœuds

- **Activité** (`pert/activity`) : `uid`, `label`, `duration`, `etp`, `charge_mode`, `charge_hours`, `responsable`, `notes`, `group`, `color`.
 - `charge_mode` (`"etp"` par défaut | `"heures"`) désigne **laquelle des deux expressions de la charge est saisie** ; l'autre est déduite via l'élongation et les paramètres de coût. Ce drapeau n'est pas cosmétique : il dit ce qui reste **invariant** quand la durée change (en `"etp"` la charge horaire suit la durée, en `"heures"` c'est l'ETP déduit qui se dilue). Stocker les deux valeurs sans lui les ferait diverger dès la première modification de durée.
 - Ne **jamais** lire `etp` / `charge_hours` en direct : passer par `pertActivityEtp` / `pertActivityHours` (`pert_engine.js`), seuls à connaître le mode. `pertActivityEtp` rend `null` quand l'ETP n'existe pas (charge en heures sur une durée nulle).
 - Un `.pert` antérieur ne porte ni `charge_mode` ni `charge_hours` : les défauts du constructeur subsistent (LiteGraph **fusionne** les propriétés sérialisées sur celles du nœud neuf) → **aucune migration**.
 - **Jalon** (`pert/milestone`) : `label`, `due_date`, `tag` (`"` | `DOTD` | `COTD` | `ING`).
 - **Label** (`pert/label`) : `text` (aucun lien, hors calcul).
-

4. Le moteur PERT (`pert_engine.js`)

Principe

- **Calcul interne en unités** (offsets depuis T0), **conversion en dates** à l'affichage. On convertit toujours l'**offset cumulé** (jamais pas-à-pas) → conversions inversibles, sans dérive.
- **Mois = mois calendaires réels** (`pertAddUnits` via `Date.setMonth`), pas un facteur fixe de 30 jours (qui dérivait sur les projets longs). Jours ($\times 1$) et semaines ($\times 7$) sont exacts.
- **Forward pass** : $ES = \max(EF \text{ des prédécesseurs })$, $EF = ES + \text{durée}$.
- **Backward pass** : $LF = \min(LS \text{ des successeurs })$, $LS = LF - \text{durée}$. **Ancre** = $\max(EF)$ (fin de projet).
- **Marge** $\text{slack} = LF - EF$. **Chemin critique** = marge **minimale** ($\text{slack} \leq \text{minSlack} + \epsilon$), ce qui reste correct même quand une échéance imposée rend des marges **négatives**.
- **Détection de cycle** par DFS tricolore avant tout calcul.

Subtilité des jalons

Le rôle d'un jalon dépend de sa **topologie** (calculé dans le forward pass) :

- **Jalon entrant** (aucun entrant + au moins un sortant + `due_date`) : $ES = EF = \text{offset}(\text{due_date})$ (planché à T0). Modélise une **contrainte externe** qui retarde la chaîne aval.
- **Jalon terminal / point de contrôle** : la `due_date` **borne le LF**, ne force pas l'ES.
- Le drapeau `target_missed` (EF calculé > cible) pilote l'alerte visuelle.

Rendu du chemin critique

`pertHighlightCriticalPath` colore les **liens** contraignants (via `link.color` , mécanisme natif LiteGraph) et mémorise l'ensemble des nœuds réellement mis en évidence dans `window.pertCriticalPathIds` — dont dérive la barre de statut (coût + nombre), pour rester cohérent avec le tracé rouge, qu'il y ait ou non une sélection.

Layout automatique

`pertAutoLayout` : packing par couloirs, abscisse $\propto ES$ (allure Gantt), tâches d'un même groupe sur des couloirs voisins, jalons terminaux en bande haute. **Déclenché manuellement** uniquement (bouton « Réorganiser ») pour ne jamais casser un placement à la main.

5. Rendu LiteGraph custom (`nodes.js` , `link_routing.js`)

LiteGraph fournit le canvas, le pan/zoom, la sélection, la sérialisation et les liens. Tout le **rendu des nœuds** est **custom** pour coller au PERT.

- **Nœuds dessinés à la main** via `onDrawForeground` / `onDrawBackground` (en-tête coloré multi-lignes, sections calculées, pastilles de tag, coin drapeau des jalons, voile d'estompage du filtre).
- **Masquage de la barre de titre** : `Constructor.title_mode = LiteGraph.NO_TITLE` (⚠ le flag d'instance `flags.no_title` **n'a aucun effet** — piège classique).
- **Positions de slots explicites** (`input.pos` / `output.pos`) puisque le titre est masqué.
- **Filtre** : voile translucide **sombre** dessiné en `onDrawForeground` (donc par-dessus le contenu et les slots). L'état de filtre `window.pertFilter` est un **état de vue non sérialisé**. Quatre natures : groupe, couleur, responsable — qui ne concernent que les Activités — et **recherche** (`type:"text"` , v0.20), la seule qui porte sur les **trois types de nœuds**, sur leur nom comme sur leurs notes, insensible à la casse et aux accents.
- **Rendu des liens** : `renderLink` est **surchargé sur l'instance** `LGraphCanvas` (sans patcher la lib). Trois styles (`meta.link_mode`) : courbe (spline natif), droit (straight natif), et **coudé** = routage **orthogonal custom** qui **contourne** les nœuds (best-effort : canal vertical, sinon bande horizontale, sinon tracé direct). Garde-fous perf : élagage spatial + dégradation au-delà de 300 nœuds. Le lien élastique de création reste une courbe native.

6. UI, historique et récupération (`ui.js` , `history.js` , `autosave.js`)

- **Menus contextuels** : `getMenuOptions` / `getNodeMenuOptions` **surchargés** sur l'instance pour remplacer les menus natifs anglais par des menus français recentrés PERT ; searchbox native neutralisée.
- **Panneau Propriétés** : reconstruit à la sélection ; helpers `buildField` , `buildCombobox` (menu déroulant custom à pastilles, fiable multi-navigateurs — le `<datalist>` natif est inadapté), `buildSelect` , `buildTextarea` , `buildReadonly` . Depuis v0.19, il est scindé en **deux onglets** — *Propriétés* (saisie) et *Synthèse* (valeurs calculées + prédécesseurs / successeurs) — sur la ligne de partage « saisi / déduit ». Le panneau inactif est **masqué, jamais retiré du DOM** (les fonctions de rafraîchissement retrouvent leurs conteneurs par id), le bouton **Supprimer** vit dans un pied fixe hors de la zone qui défile, et l'onglet consulté est mémorisé entre deux sélections. Le voisinage est lu via `pertBuildAdjacency` (moteur) : une seule source de vérité avec le calcul PERT.
- **Undo/Redo** : historique par **snapshots** (`meta + graph.serialize()`), restaurés par `configure()` — même mécanisme que la persistance, donc exhaustif. Coalescence des frappes.
- **Sauvegarde automatique** : en `file://` , impossible d'écrire un fichier silencieusement → un **snapshot de récupération dans `localStorage`** , écrit périodiquement tant qu'il reste du travail

non sauvegardé, proposé à la restauration au démarrage. Activée par défaut.

- **Gestion d'erreurs** : `showToast` / `showError` / `guardUI` + filet global — indispensable en `file://` où l'utilisateur n'a pas la console.

Filtre et recherche (`window.pertFilter`)

Cinq natures de filtre — `group`, `color`, `responsable`, `progress` (qui ne « matchent » que des Activités) et `text` (nom **et** notes, sur les trois types de nœuds, insensible à la casse et aux accents). Un filtre **n'enlève rien** : il **estompe** les nœuds hors sélection sous un voile, pour que le planning reste lisible dans son ensemble.

C'est un **état de vue** : jamais sérialisé dans le `.pert`, au même titre que l'onglet courant du panneau ou de la synthèse. Une recherche **remplace** le filtre courant (ils partagent une seule variable), d'où l'obligation de resynchroniser les marqueurs d'état de la liste — sans quoi l'IHM annonce un filtre qui n'est plus actif.

Fenêtres de rapport (`synthesis.js`, `suivi.js`)

Le bouton **Synthèse** ouvre un menu à deux entrées, deux fenêtres au même patron :

- **Synthèse de planification** — le planning **tel qu'il est prévu** : vue d'ensemble, jalons entrants / sortants, agrégats par groupe (charge, coût, fin au plus tard), et un onglet **Analyse**. Ses quatre onglets deviennent **quatre chapitres** à l'impression.
- **Suivi d'avancement** — le même planning **confronté à la date du jour**. Il ne recalcule rien. La date du point est surchargeable (`window.pertSuiviToday`) : sans cette couture, ni test ni capture ne seraient reproductibles.

Deux principes structurants. **L'onglet Analyse est fait pour s'enrichir** : un contrôle est un objet `{ id, title, hint, columns, rows }` poussé dans une liste, le rendu est générique ; un contrôle **sans anomalie n'est pas affiché** (une liste de choses à regarder, pas des cases vertes à faire défiler) et chaque ligne doit **mener au(x) nœud(s)** concerné(s). **La coquille de fenêtre est mutualisée** par des **classes** (`.synth-content`, `.synth-tabs`, `.synth-printing`) et non par les ids de la synthèse — deux fenêtres coexistant, une règle `@media print` qui ciblerait un id force-afficherait la fenêtre fermée sur le papier.

7. Import Excel legacy (`import_excel.js`)

Le `.xlsm` est un ZIP dont **toute la donnée utile est dans** `xl/drawings/` (les groupes de formes = nœuds, les connecteurs = liens ; les cellules sont cosmétiques sauf l'onglet **MANUEL** de configuration : feuille cible, T0, unité). Traitement 100 % `file://` :

- **Dézip par fflate** (`unzipSync`), parsing **DOMParser** natif, `<input type="file">` + `FileReader.readAsArrayBuffer` , **jamais fetch** .
 - Convention de nommage : **A** =activité, **S** =jalón, **E** =**jalón d'entrée** (matérialisé en jalón entrant avec ses arêtes). La couche **transforms purs** (`buildImportModel`) est séparée de la couche DOM/ZIP → testable en Node.
-

8. Exports (`export*.js`)

Un **seul bouton** ouvre une fenêtre listant les formats (liste data-driven `PERT_EXPORT_FORMATS` + `pertRegisterExportFormat` , triée par `order`).

- **PNG / PDF** : **rendu hors-écran** indépendant du zoom (un `LGraphCanvas` temporaire calé sur la boîte englobante, fond blanc, un seul `draw`), puis `toDataURL` / `jsPDF` (A4, `compress:true`).
 - **CSV** : dump brut, séparateur `;` , décimales `,` , BOM UTF-8.
 - **XLSX** : un `.xlsx` est un **ZIP de XML** → **mini-writer maison sur fflate** (`export_xlsx.js` , `pertXlsxBuild`) : cellules texte/nombre/date/formule, styles (formats date, `0.00` , fills de couleur, gras) déduplicués, `sharedStrings` . **Pas de SheetJS** (Apache-2.0, exclu par « MIT uniquement »).
 - **Gantt chargé** et **MS Project (MSPDI XML)** partagent `pertScheduleModel()` (tri groupes par ES précoce, classement jalons entrée/sortie, colonnes de périodes, liens). Aucune bibliothèque `.mpp` native n'existant côté navigateur (MIT/offline), MS Project est produit en **MSPDI XML** écrit à la main.
 - Téléchargements via `pertDownloadBlob` (objet URL, fonctionne en `file://`).
-

9. Packaging (`scripts/build-bundle.js`)

Le développement se fait sur la structure `index.html` + `src/` + `lib/` . Un **script Node natif sans dépendance** produit le **livrable autonome** `dist/pertflow.html` en **inlinant** les `<link>` / `<script>` (avec garde-fou : échec s'il reste une référence `lib/` / `src/` / `css/`). Il injecte `window.PERTFLOW_BUILD = { date, tag }` (lu par la popup « À propos »). Le bundle est **versionné** et régénéré en fin de session.

10. Récapitulatif des choix et de leurs raisons

Choix	Raison
Pas de modules ES6, tout en global via <code><script src></code>	Ouverture <code>file://</code> sans serveur (CORS)
<code>FileReader</code> au lieu de <code>fetch</code>	Lecture de fichiers locaux en <code>file://</code>
fflate pour lire et écrire les Excel	MIT, déjà nécessaire à l'import ; évite SheetJS (Apache-2.0)
MS Project en MSPDI XML	Aucune lib <code>.mpp</code> native MIT/offline en navigateur
Valeurs PERT recalculées, non sérialisées	Cohérence garantie au chargement
Rendu de nœuds/liens custom sur l'instance	Coller au PERT sans patcher LiteGraph
Snapshots pour l'undo et l'autosave	Un seul mécanisme robuste et exhaustif
Autosave en <code>localStorage</code>	Seul stockage persistant possible en <code>file://</code>
Layout manuel (jamais auto)	Ne jamais casser un placement à la main
Filtre et onglets = état de vue , hors <code>.pert</code>	Un fichier décrit un planning, pas une session de travail
Avancement hors de tout calcul	Le PERT reste l'objectif : le renseigner ne doit rien déplacer
Charge : un mode de saisie, pas deux valeurs libres	Le mode dit l'invariant quand la durée bouge ; deux valeurs stockées sans lui divergeraient
Filtre qui estompe au lieu de masquer	Garder le planning lisible dans son ensemble